Overview

> *Version of 2026-07-06*

This project can be done *alone* or as a *pair*. It makes up 40% of your overall grade.

The project you are tasked with can be split into two parts:

- required features and
- elective features

All required features have to be implemented, and their points are part of the grading.
You have to choose a certain amount of the elective features, and implement them accordingly.
If you choose to work as a pair the amount of elective features required changes accordingly – refer to the appropriate section.

> **Updated Information:**
>
> The project presentations will happen on **Mon. 2026-09-07**.
> The project has to be handed in a week later until **Mon. 2026-09-14**.
> The hand in is in form of a ZIP Archive.
> If the ZIP Archive is bigger than what Moodle allows, you can hand in a link which allows us to download the Archive. *This Link has to be available at least until the end of the semester.*

Required Features

> *You have to fulfil **all** of these, independent of if you work solo or as a pair.*

- Your project has to be **runnable with** `uv`. Either as a *self-contained script* or via `uvx`.
  - Refer to the documentation of `uv` on details on how to achieve this.
- **A user interface.** You can choose any UI framework that runs on Windows, MacOS, and Linux.
  - This could be a graphical interface using for example *marimo + wigglestuff* or *NiceGUI* or
  - a textual interface using for example *Textual*.
  - If you elect to use a textual interface the requirements for the features *stay the same*, so keep that in mind.
    - Pro: Using a textual interface will make using the cluster easier – and will look sleek as hell.
    - Con: Some stuff might be harder / more custom code to represent.
- **A chat interface**.
  - It needs to support user input by sending text-messages.
  - The conversation history needs to be visible to the user.
  - The user needs to be able to view *any* part of the prior conversation history (scrolling / paging behaviour).
- The ability to **start, stop, and resume chats**.
  - The interface needs to support the switching between conversations.
  - The interface needs to allow the user to create *new*, *continue old*, and *remove old* conversations.
- The **ability to switch** the underlying **LLM model** used for generation.
  - The interface needs to support the loading of at least *two* different models for generation.
- A **simple memory mechanism** for recalling prior information from *groups* of chats.
  - Similar to the "Project Folder" feature of ChatGPT / Claude.

- Add **at least *two* fine-tuned models** for special purposes (Full / LoRA / QLoRA).
  - We would recommend using LoRA/QLoRA, as their training is more efficient, and they are easier to exchange (lightweight).
- **Demonstrate** the usage of **your project** with a video demo.
  - Demonstrate the required & elective features you have implemented.
  - Demonstrate at least one real-world use-case that benefitted from your elective features.

Elective Features

> *You are required to choose **2 (two)** out of these. If you work as a pair you will have to choose **4 (four)** instead.*

- Add the ability to use adaptive Retrieval Augmented Generation
  - It needs to follow the same diagram as shown in the slides, using a judge, rewriter, multiturn retrieval and summarization or an equivalently complex approach.
  - It needs to support user provided data, i.e. the user should be able to provide their own documents, at least as text files and pdf's.
- Add *extensible* tool-calling, where the tools can be provided by the user.
  - The user should be able to provide a "tool-definition" on the fly, that the model can later use for calling upon that tool.
    - This could be something like:

```
{
  "name": "summarizing",
  "type": "shell",
  "command": "uv python summarization.py",
  "args": [
      {"name":"text", "type":"text"}
  ]
}
```

- Add sub-agent deployment, where a sub-agent can fulfill a LLM defined task.
  - This can be realized as a tool call, where the model calls upon another LLM to *do* a certain task.
  - Extra credit if the controlling LLM can continue onwards without having to wait for the sub-agent.
- Add the ability to *securely* run code generated by the system and make use of the outputs.
  - The code generated should be able to use only a safe subset of the stdlib of python, as well as common data manipulation, ML, and plotting libraries.
  - The code should *never* be able to change files outside it's designated location.
  - Think about sandboxing, containerization and how to make sure the model has to follow safe defaults.
- Add the ability to *search the web* repeatedly and self-controlled to improve the LLMs generation.
  - Make sure the web-searching capabilities go further than just using the direct results form DuckDuckGo or Google.
  - The model *should* be able to traverse the internet at large, meaning that it should be able to *find* and *visit* a page then make use of the information within.
- Add the ability to do preference optimisation via human feedback.
  - Sometimes the user should presented with a choice of generated texts, and choose which they would prefer.
  - You should also provide code to train a LoRA / QLoRA on this choice data, such that the model can be *continuously* improved.
- Add the ability to ingest multimodal data, especially but not limited to images.
  - The model should be able to make use of most file-types: pdf, images, audio, text, word, excel,...

    - If you elect this we expect the use of a *inherently* multimodal model for generation (a Vision+Language Model).
- Add *concurrent multi-user* support.
    - Make it so that you can support more than one user, each with their own chat's, projects, memory etc.
    - Make sure that the users can independently and *concurrently* generate answers.
- Add intelligent context management.
    - We want you to implement at least *context compression* and *context isolation*, as well as *context selection*.
- Add advanced prompt caching methods, by implementing one method (like H2O) yourself.
- Add the "Tree-of-Thought" reasoning approach to your model.
    - Refer to the Paper for details.
    - This would be a *user enabled* "thinking" mode.